Compte rendu d’utilisation de l’IA

Guide pour les étudiants • Projet Python L2 • 2026–2027

Ce que j’attends dans votre PDF

Vous pouvez utiliser l’intelligence artificielle pour vous aider dans votre projet. Votre compte rendu doit montrer comment vous l’avez utilisée, ce que vous avez compris et ce que vous avez réellement intégré dans votre code.

Pour **chaque prompt utilisé dans le projet**, expliquez son objectif, reproduisez la demande exacte, présentez la réponse utile de l’IA et le code fourni, puis décrivez vos adaptations, l’emplacement de l’intégration et vos vérifications. Documentez aussi les demandes d’explication, de débogage et les propositions que vous avez écartées.

Organisation du document à rendre

1. Présentation. Nom(s), groupe, titre du projet, objectif du programme, outil d’IA utilisé et modèle si connu. Précisez les dates des échanges.

2. Journal des prompts. Numérotez les échanges dans l’ordre. Utilisez la fiche de la page suivante pour chaque prompt, y compris les relances et corrections.

3. Bilan personnel. Expliquez ce que l’IA vous a aidé à comprendre, les erreurs rencontrées, les limites de ses réponses et ce que vous avez réalisé vous-même.

4. Annexes si nécessaire. Joignez les réponses longues ou extraits complémentaires, avec un renvoi depuis la fiche concernée. Dans le corps du rapport, gardez les passages nécessaires pour comprendre vos choix.

Ce qui permet d’évaluer votre démarche

La traçabilité des échanges, la compréhension du code, la justification de vos adaptations et les preuves de vérification. Un code qui fonctionne ne suffit pas : vous devez pouvoir expliquer son rôle et son fonctionnement.

Exigence de fond : chaque choix doit être justifié par une contrainte du projet, une explication technique ou un résultat vérifiable. Les formulations « j’ai copié le code », « l’IA a corrigé » ou « cela fonctionne » ne constituent pas une analyse suffisante.

Rendu attendu : un PDF lisible, avec du code sous forme de texte, accompagné du code final du projet selon les modalités de remise du cours.

Une fiche pour chaque prompt

Copiez cette structure autant de fois que nécessaire. Écrivez les explications avec vos propres mots et reliez les relances au prompt d'origine.

1. Identification

Prompt n°…, date, outil et modèle si connu. Fonctionnalité concernée.

2. Besoin et objectif

Décrivez l'état du code avant la demande, le problème précis, votre première tentative et ce qui vous bloque. Formulez le comportement attendu et les contraintes que la réponse devra respecter.

3. Prompt exact

Reproduisez le texte envoyé à l'IA, avec le contexte et le code transmis. Ne le reformulez pas après coup.

4. Réponse et code proposés

Présentez la réponse utile et le code reçu. Si aucun code n'est fourni, indiquez-le et expliquez l'apport de la réponse. Renvoyez en annexe si nécessaire.

5. Compréhension et choix

Expliquez chaque ligne de logique ajoutée ou modifiée : rôle des variables, types, conditions, boucles et valeurs retournées. Analysez les hypothèses, cas limites et erreurs possibles. Justifiez chaque élément retenu, modifié ou refusé.

6. Intégration dans votre code

Indiquez le fichier, la classe et la fonction concernés, ainsi que les lignes dans la version finale remise. Montrez le code avant et après. Distinguez le code de l'IA de vos modifications et justifiez chacune d'elles. Expliquez les imports, appels et interactions avec le reste du programme.

7. Vérifications et résultat

Décrivez les essais réellement effectués, leurs données, le résultat attendu et le résultat observé. En cas d'échec, expliquez la correction ou renvoyez au prompt suivant.

Si vous n'avez pas intégré une proposition : écrivez « non intégrée » et justifiez ce choix. N'inventez ni échanges ni résultats de tests.

Exigences de justification

Une trace complète et vérifiable

Conservez tous les prompts et les réponses liés au projet, y compris les essais infructueux et les corrections. Placez les échanges complets en annexe lorsqu'ils sont longs et associez-les à leurs fiches par un identifiant. Mentionnez les fichiers ou extraits transmis à l'IA. Pour un travail en groupe, précisez qui a effectué la demande et qui a intégré ou vérifié le résultat.

Une analyse personnelle du code

Pour chaque proposition, confrontez la réponse aux consignes et à l'architecture du projet. Expliquez pourquoi elle convient ou pourquoi elle échoue. Décrivez au moins une autre approche lorsqu'un choix technique réel existe et justifiez votre choix en termes de clarté, de robustesse ou de coût. Pour une opération répétée sur beaucoup de données, discutez aussi le coût en temps ou en mémoire lorsque cela est pertinent.

Vous devez pouvoir expliquer le code retenu sans relire la réponse de l'IA, prévoir son résultat sur une entrée donnée et identifier les endroits à modifier si la consigne évolue. Une explication générée par l'IA doit être vérifiée et reformulée avec votre propre compréhension.

Des vérifications reproductibles

Pour chaque fonctionnalité modifiée, présentez un cas normal, les cas limites pertinents et un cas d'échec lorsque celui-ci peut se produire. Donnez les données d'entrée, les étapes ou commandes permettant de reproduire l'essai, le résultat attendu, le résultat observé et votre conclusion. Justifiez un cas non applicable.

Joignez une preuve adaptée : sortie de console, assertions exécutées, message d'erreur ou capture de l'interface. Après une correction, rejouez le cas qui échouait et vérifiez une fonctionnalité existante susceptible d'être affectée. Une capture sans explication ou une liste de tests non exécutés ne suffit pas.

Un bilan individuel argumenté

Identifiez votre contribution personnelle, les notions apprises et les difficultés qui restent. Analysez au moins une limite réellement rencontrée dans les réponses de l'IA ; si vous n'en avez identifié aucune, expliquez vos contrôles et les incertitudes qui subsistent. Reliez le bilan à des prompts et à des extraits précis du projet.

Un compte rendu est incomplet si les prompts exacts, le code final, les raisons des modifications ou les preuves de vérification manquent. Si un point n'a pas été vérifié, indiquez-le explicitement et expliquez la conséquence pour votre projet.

Exemple concret : Typosteroides

Exemple pédagogique fictif. Les échanges et résultats ci-dessous illustrent le compte rendu attendu ; ils ne sont pas l'historique réel d'un étudiant. Le contexte vient de **TD2/td2_eleve.py**, classe **Jeu**, méthode **precision()**.

Prompt n°1 — Demander un calcul de précision

Objectif : remplacer la valeur fixe de la méthode par le pourcentage de frappes correctes. L'affichage du jeu appelle déjà cette méthode.

Prompt envoyé : « Dans mon jeu Python, la classe Jeu possède self.frappes_correctes et self.frappes_totales. La méthode precision(self) renvoie actuellement 100.0. Propose une méthode qui renvoie le pourcentage de frappes correctes et explique la formule. »

Code proposé par l'IA dans cet exemple :

```
def precision(self):
    return 100 * self.frappes_correctes / self.frappes_totales
```

Ma compréhension : je divise le nombre de frappes correctes par le nombre total de frappes, puis je multiplie par 100. Avec 8 frappes correctes sur 10, la précision vaut 80 %.

Mon analyse : au début d'une partie, les deux compteurs valent zéro. La formule provoque alors une division par zéro. Cette réponse ne respecte pas encore le TODO 10a, qui demande 100 % lorsqu'aucune frappe n'a été effectuée. Je ne conserve donc pas ce code tel quel.

Prompt n°2 — Corriger le cas initial

Objectif : compléter la réponse précédente pour respecter la consigne du TD.

Prompt envoyé : « Ta méthode divise par zéro au début de la partie car self.frappes_totales vaut 0. Le TODO 10a exige de renvoyer 100.0 si aucune frappe n'a été effectuée. Corrige uniquement precision(self) et explique le rôle du if. »

Code corrigé proposé par l'IA :

```
def precision(self):
    if self.frappes_totales == 0:
        return 100.0
    return 100 * self.frappes_correctes / self.frappes_totales
```

Ma compréhension : le if traite le cas sans frappe avant la division. Le return termine la méthode immédiatement dans ce cas. Les 100 % sont une convention du TD pour le début de partie.

Exemple : intégration et validation

Ce que j’ai changé dans mon programme

Emplacement : TD2/td2_eleve.py → classe Jeu → méthode precision(), TODO 10a (vers la ligne 364 dans la version fournie). Je remplace uniquement le corps de cette méthode et je garde son nom pour que les appels existants fonctionnent.

Avant :

```
def precision(self):
    """Pourcentage de frappes correctes."""
    # TODO 10a : 100 * correctes / totales (100 si aucune frappe)
    return 100.0
```

Après : code final intégré dans la classe Jeu

```
def precision(self):
    """Pourcentage de frappes correctes."""
    if self.frappes_totales == 0:
        return 100.0
    return 100 * self.frappes_correctes / self.frappes_totales
```

Mes adaptations : je garde la documentation existante, je retire le TODO résolu et j’indente la méthode dans la classe. Aucun import n’est nécessaire. L’affichage utilise déjà self.precision() ; je n’ajoute pas de nouvel appel.

Vérifications à documenter

Cas à reproduire : 8 frappes correctes sur 10 doivent donner 80.0 %. Sans frappe, le résultat doit être 100.0 % sans exception. Avec 10 bonnes frappes sur 10, il doit être 100.0 % ; avec 0 sur 5, il doit être 0.0 %. Dans votre rapport, donnez le protocole et les résultats réellement observés.

Vérification dans le jeu : lancer une partie, vérifier l’affichage initial, puis effectuer des frappes correctes et incorrectes. Consigner ce qui s’affiche et vérifier que les compteurs évoluent correctement. Ce contrôle complète les essais de calcul.

Exemple de bilan personnel

« L’IA m’a aidé à écrire la formule. J’ai repéré un cas oublié, demandé une correction et intégré la méthode dans ma classe. J’ai compris le rôle du test avant la division. Cette fonction calcule un pourcentage ; elle suppose que les compteurs sont mis à jour correctement ailleurs dans le programme. »